

Computer System Design & Application

计算机系统设计与应用A

陶伊达 (TAO Yida)

taoyd@sustech.edu.cn

An abstract graphic on the left side of the slide, featuring concentric circles and various digital patterns like binary code and data streams in shades of blue and green.

Lecture 8

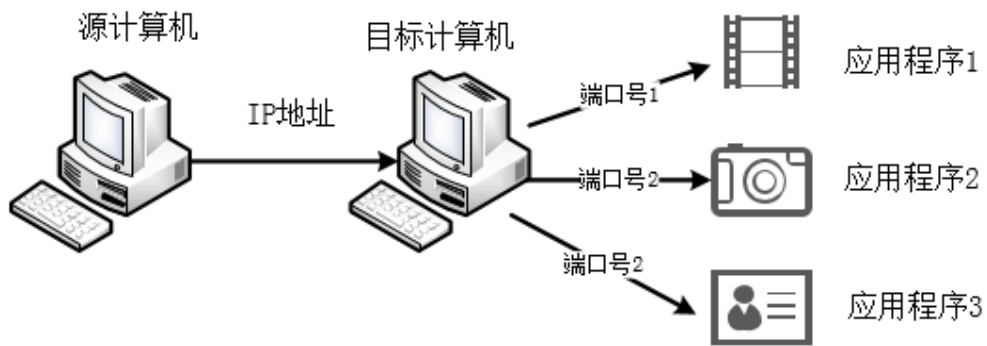
- Socket Programming
- Getting Web Data



Networking

Networking is a concept of connecting two or more computing devices together so that we can share resources

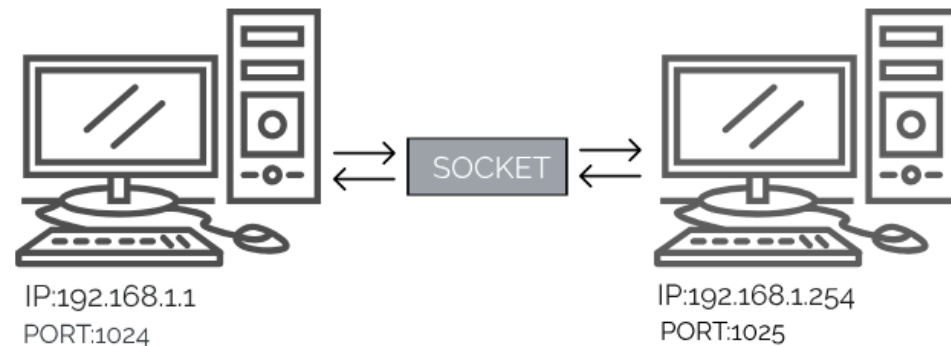
Networking Terminology



- **IP address:** a unique address that distinguishes a device on the internet or a local network
- **Domain name:** a human-friendly version of an IP address that you enter in browser (translated by DNS)
- **Port number:** a number used to identify different applications/processes uniquely

Socket

- To communicate, a client program and a server program establish a connection to one another
- Each program binds a **socket** to its end of the connection
- A socket is one **endpoint** of a two-way communication link between two programs running on the network.
 - Endpoint: IP address + Port number
- To communicate, the client and the server each reads from and writes to the socket bound to the connection.



<https://examradar.com/java-networking-network-basics-socket-overview/>

Socket

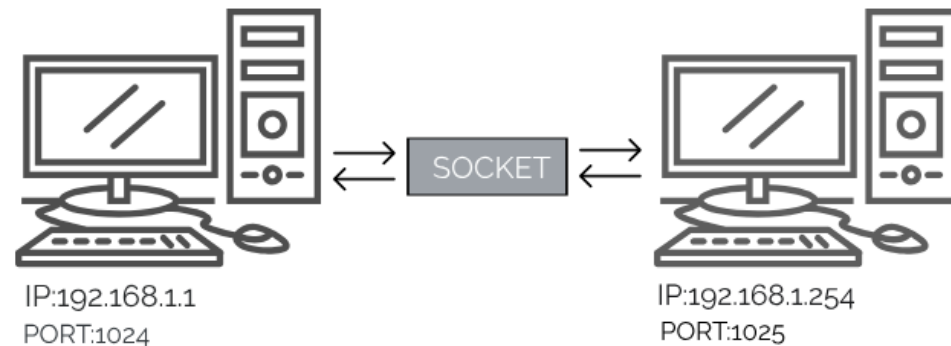
- The `java.net` package provides a powerful and flexible infrastructure for networking, providing various classes and interfaces that execute the low-level communication features

```
Socket(String host, int port)
```

Creates a stream socket and connects it to the specified port number on the named host.

```
ServerSocket(int port)
```

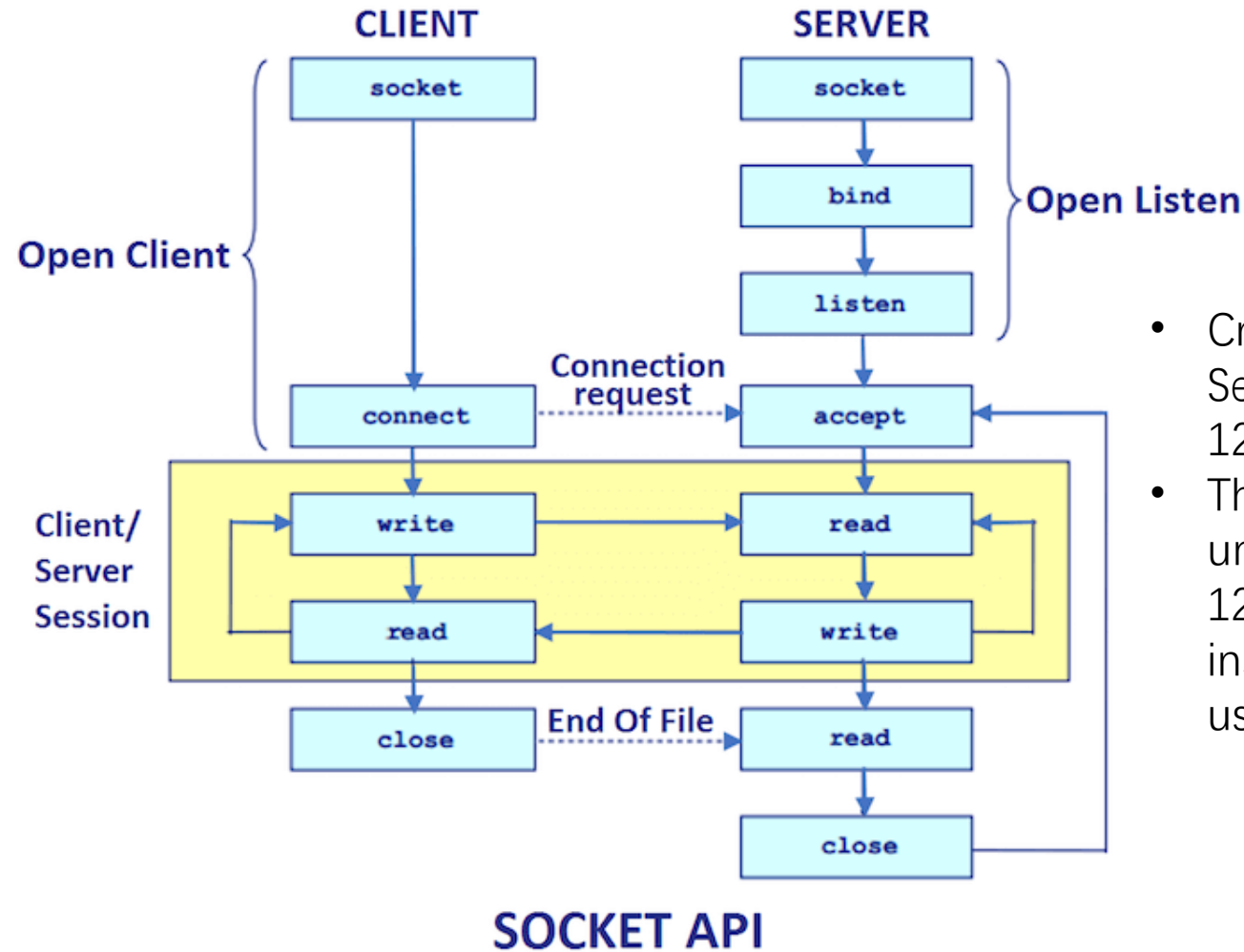
Creates a server socket, bound to the specified port.



<https://examradar.com/java-networking-network-basics-socket-overview/>

```
Socket s = new Socket("www.serverip.com", 1234);
```

```
ServerSocket ss = new ServerSocket(1234);  
Socket s = ss.accept();
```

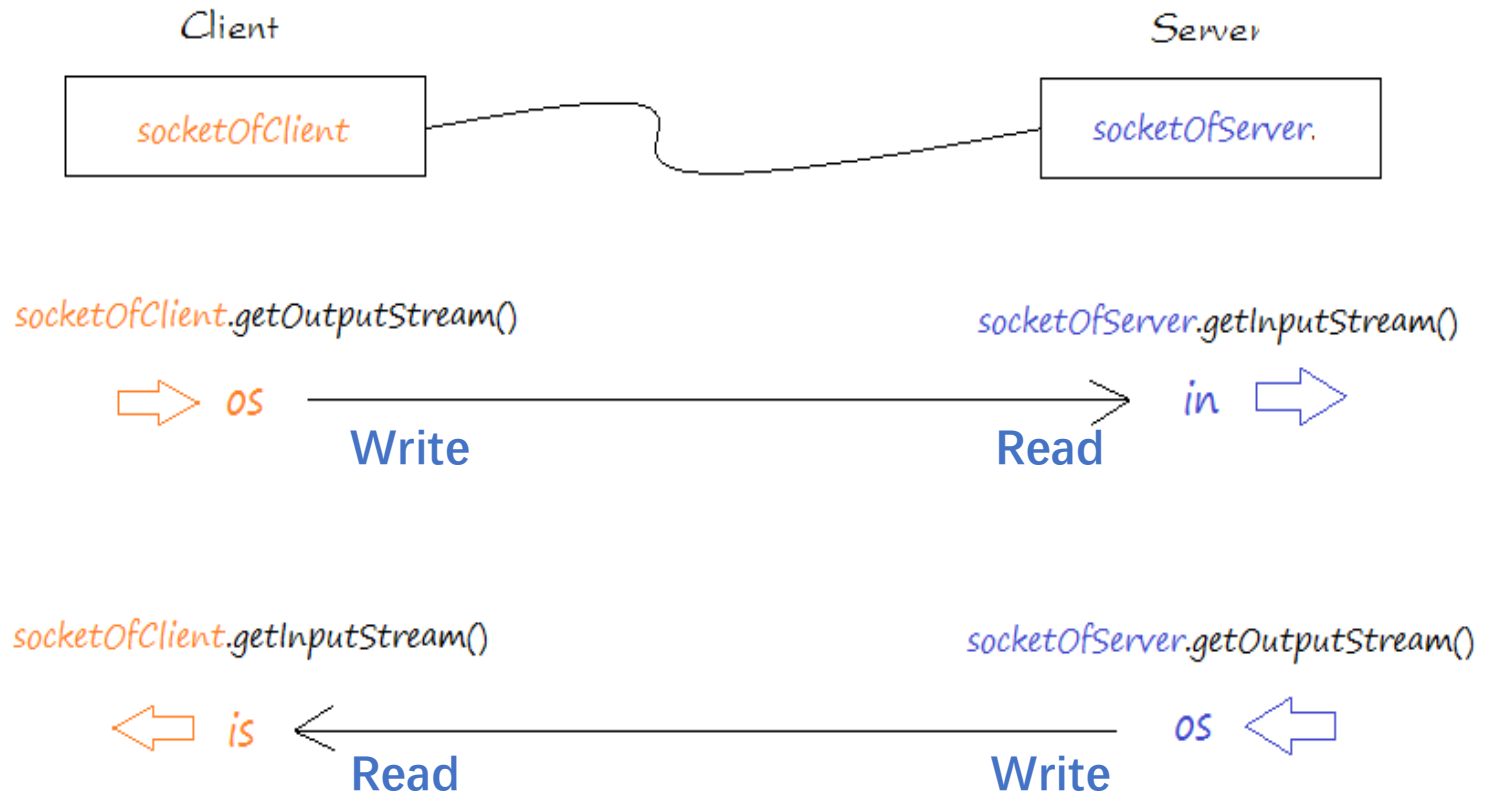
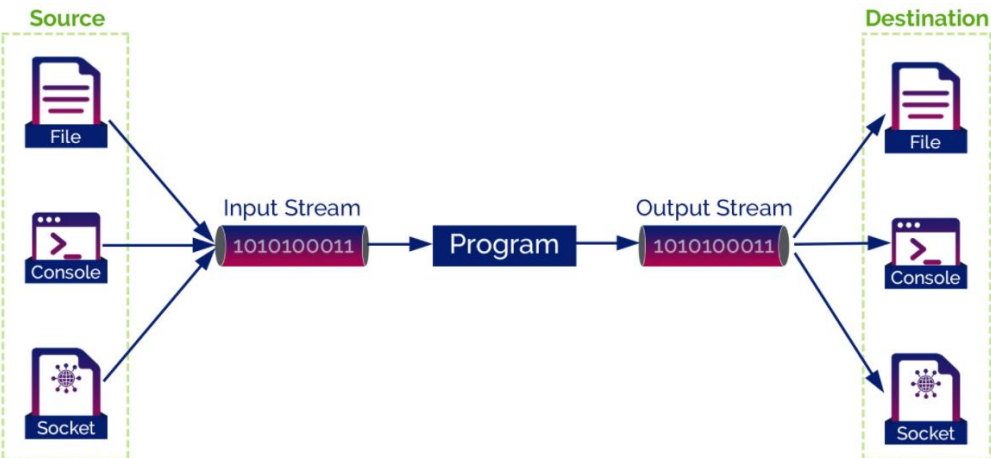


- Create an instance of Socket by passing the IP or hostname of the server and a port number
- If the connection fails, an Exception is thrown
- Otherwise, establish the connection and use Socket to read and write.

- Create an instance of ServerSocket by binding to 1234 port number
- The `accept()` method **waits** until a client connects to port 1234, and if so, return an instance of Socket that is used for reading and writing.

Reading from and Writing to a Socket

- After establishing the connection, we can use `socket.getInputStream()` and `socket.getOutputStream()` for both the client and the server



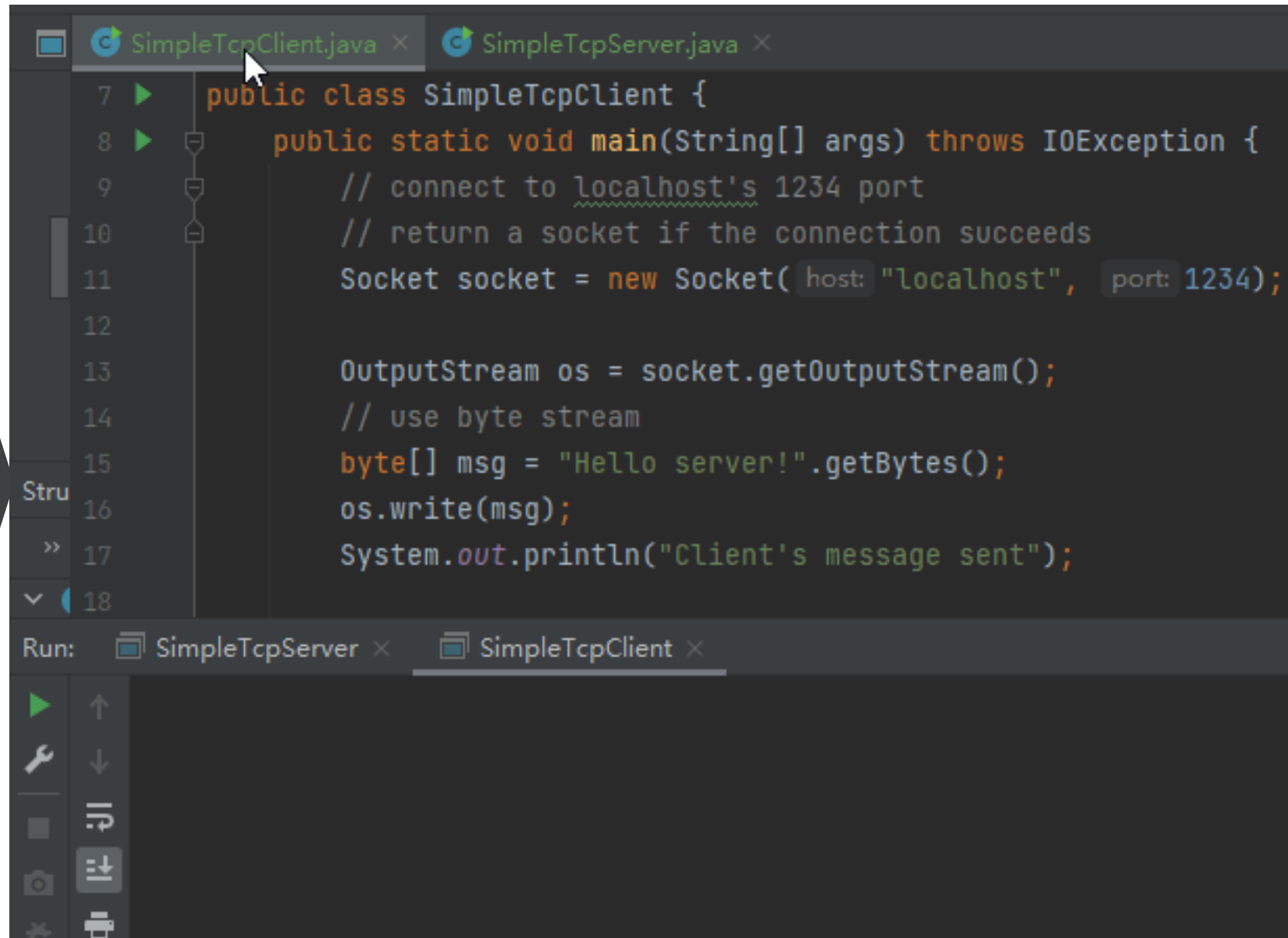
A Toy Example: Client

```
public class SimpleTcpClient {  
    public static void main(String[] args) throws IOException {  
        // connect to localhost's 1234 port  
        // return a socket if the connection succeeds  
        Socket socket = new Socket("localhost", 1234);  
  
        // Get OutputStream  
        // and write messages as bytes  
        OutputStream os = socket.getOutputStream();  
        byte[] msg = "Hello server!".getBytes();  
        os.write(msg);  
        System.out.println("Client's message sent");  
  
        // closing the OutputStream will close the associated socket.  
        os.close();  
    }  
}
```

A Toy Example: Server

```
public class SimpleTcpServer {  
    public static void main(String[] args) throws IOException {  
        // Listen to port 1234  
        ServerSocket serverSocket = new ServerSocket(1234);  
  
        // accept() blocks until a client connects  
        // if a client connects successfully, return a Socket instance  
        System.out.println("Waiting for client.....");  
        Socket socket = serverSocket.accept();  
        System.out.println("Client connected.");  
  
        // use the socket's inputStream to read message from the client  
        InputStream inputStream = socket.getInputStream();  
  
        // get client msg as bytes and print it  
        byte[] buf = new byte[1024];  
        int readLen = 0;  
        while((readLen = inputStream.read(buf)) != -1){  
            System.out.println(new String(buf, 0, readLen));  
        }  
  
        // closing the InputStream will close the associated socket  
        inputStream.close();  
        serverSocket.close();  
    }  
}
```

A Toy Example

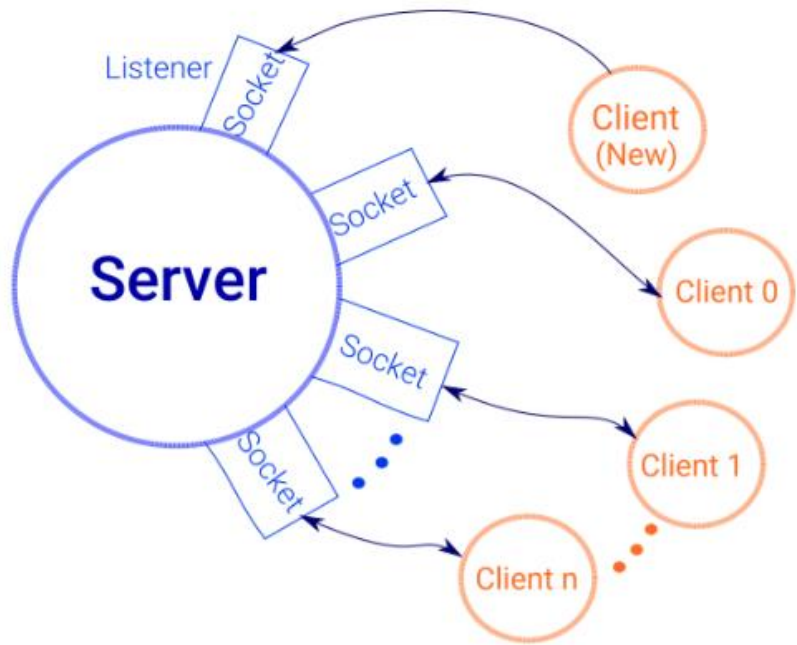


The screenshot shows an IDE with two tabs: SimpleTcpClient.java and SimpleTcpServer.java. The SimpleTcpClient.java tab is active, displaying the following Java code:

```
7 public class SimpleTcpClient {  
8     public static void main(String[] args) throws IOException {  
9         // connect to localhost's 1234 port  
10        // return a socket if the connection succeeds  
11        Socket socket = new Socket( host: "localhost", port: 1234);  
12  
13        OutputStream os = socket.getOutputStream();  
14        // use byte stream  
15        byte[] msg = "Hello server!".getBytes();  
16        os.write(msg);  
17        System.out.println("Client's message sent");  
18    }  
}
```

Below the code editor, there is a 'Run:' section with two tabs: SimpleTcpServer and SimpleTcpClient. The SimpleTcpClient tab is selected. To the left of the tabs are several icons: a green play button, a wrench, a square, a camera, and a magnifying glass.

Why “Toy” Examples?



- The toy server reads only 1 message then exits; In practice, server keeps running
- The toy client/server handles byte directly, which is cumbersome
- In practice, servers need to support multiple clients at the same time

More practical: use multi-threads on server side: whenever a client request comes, a separate thread is assigned for handling each request

Case Study: Banking Service

```
BankAccount
├── BankAccount()
├── BankAccount(double)
├── deposit(double): void
├── withdraw(double): void
├── getBalance(): double
└── balance: double
```

- A bank account has a balance that can be changed by deposits and withdrawals.

```
public synchronized void deposit (double amount) {
    balance = balance + amount;
    notifyAll();
}
```

```
public synchronized void withdraw (double amount) {
    try {
        while (balance < amount) wait();
        balance = balance - amount;
    } catch (InterruptedException e) {}
}
```

deposit() and withdraw() are properly synchronized

Case Study: Banking Service

	Bank
	Bank(int)
	deposit(int, double): void
	withdraw(int, double): void
	getBalance(int): double
	accounts: BankAccount[]

A bank has multiple bank accounts

```
public class Bank {  
    private BankAccount[] accounts;  
  
    /**  
     * Constructs a bank account with a given  
     * number of accounts.  
     * @param size the number of accounts  
     */  
    public Bank (int size) {  
        accounts = new BankAccount[size];  
        for (int i = 0; i < accounts.length; i++) {  
            accounts[i] = new BankAccount();  
        }  
    }  
}
```

Case Study: Banking Service

	Bank
	Bank(int)
	deposit(int, double): void
	withdraw(int, double): void
	getBalance(int): double
	accounts: BankAccount[]

A bank can withdraw from or deposit to a specific account

```
public void deposit (int accountNumber, double amount) {  
    BankAccount account = accounts[accountNumber];  
    account.deposit( amount);  
}
```

```
public void withdraw (int accountNumber, double amount) {  
    BankAccount account = accounts[accountNumber];  
    account.withdraw( amount);  
}
```

```
public double getBalance (int accountNumber) {  
    BankAccount account = accounts[accountNumber];  
    return account.getBalance();  
}
```


Banking Service Protocol

Table 2 A Simple Bank Access Protocol

Client Request	Server Response	Description
BALANCE n	n and the balance	Get the balance of account n
DEPOSIT n a	n and the new balance	Deposit amount a into account n
WITHDRAW n a	n and the new balance	Withdraw amount a from account n
QUIT	None	Quit the connection

Whenever you develop a client/server application, you need to specify some application-level protocol (协议) that clients can use to interact with the server

Case Study: Banking Service

Ⓢ	📁	BankService
Ⓜ	📁	BankService(Socket, Bank)
Ⓜ	📁	run(): void
Ⓣ	🔒	socket: Socket
Ⓣ	🔒	bank: Bank

A bank service executes the banking service protocol

```
public class BankService implements Runnable {  
    private Socket socket;  
    private Bank bank;  
  
    public BankService (Socket socket, Bank bank) {  
        this.socket = socket;  
        this.bank = bank;  
    }  
}
```

Case Study: Banking Service

		BankService
		BankService(Socket, Bank)
		run(): void
		socket: Socket
		bank: Bank

```
public void run() {
    Scanner in = new Scanner( socket.getInputStream());
    PrintWriter out = new PrintWriter( socket.getOutputStream());

    while (true) {
        if (!in.hasNext()) return;
        String command = in.next();
        if ("QUIT".equals(command)) return;

        int account = in.nextInt();
        double amount;
        switch (command) {
            case "DEPOSIT" :
                amount = in.nextDouble();
                bank.deposit( account, amount);
                break;
            case "WITHDRAW" :
                amount = in.nextDouble();
                bank.withdraw( account, amount);
                break;
            case "BALANCE" :
                break;
            default:
                out.println( "Invalid command" );
                out.flush();
                return;
        }
        out.println( account + " " + bank.getBalance( account) );
        out.flush();
    }
}
```

Table 2 A Simple Bank Access Protocol

Client Request	Server Response	Description
BALANCE n	n and the balance	Get the balance of account n
DEPOSIT n a	n and the new balance	Deposit amount a into account n
WITHDRAW n a	n and the new balance	Withdraw amount a from account n
QUIT	None	Quit the connection

Bank Server

```
public class BankServer {
    public static void main (String[] args) throws IOException {
        // initialized bank with 10 accounts
        Bank bank = new Bank(10);

        ServerSocket server = new ServerSocket(8888);

        System.out.println( "Waiting for clients to connect..." );
        // server keeps running
        while (true) {
            Socket socket = server.accept();
            System.out.println( "Client connected." );

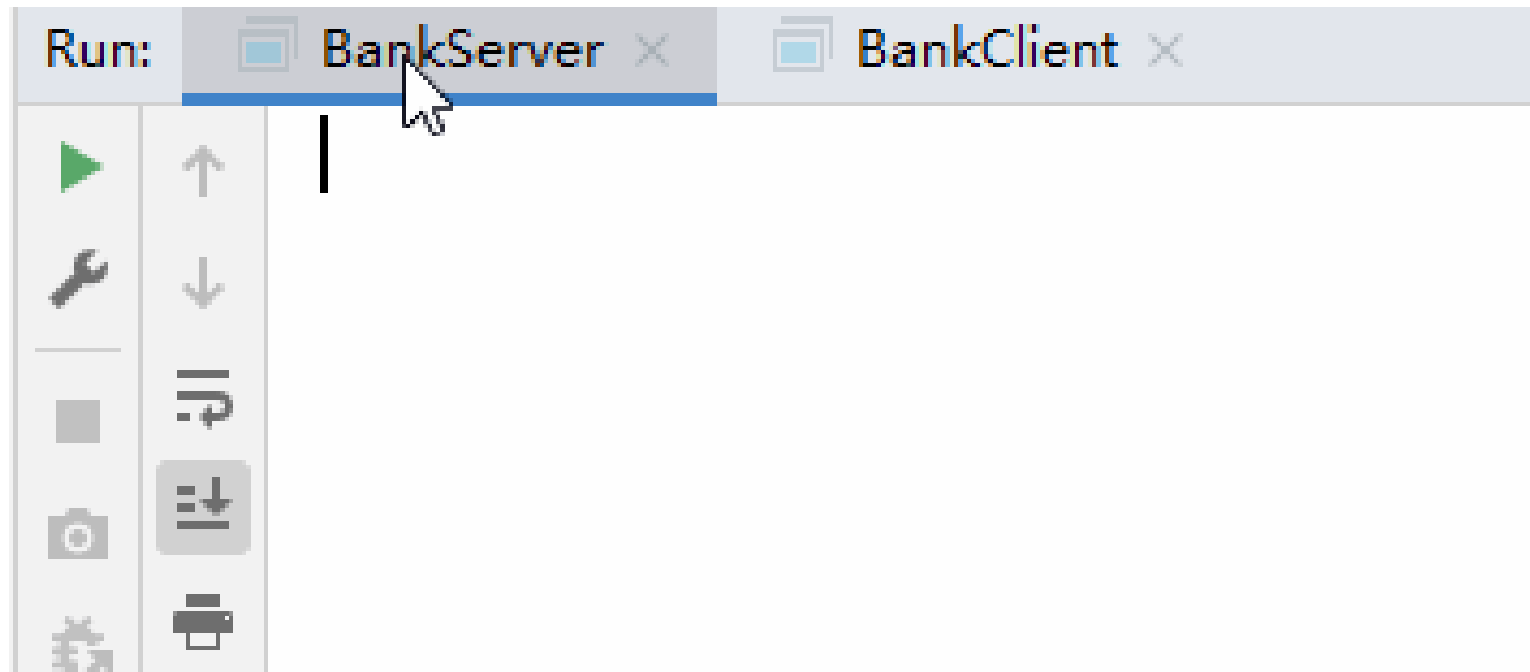
            // start a thread for providing service to the client
            BankService service = new BankService(socket, bank);
            Thread t = new Thread(service);
            t.start();
        }
    }
}
```

Bank Client

- To communicate with the server by sending and receiving text, you could turn the streams into **Scanners** and **PrintWriters**
- Use **println** instead of **print** to mark the end of input
- The **flush()** method empties the buffer and forwards all waiting characters to the destination.

```
public class BankClient {  
    public static void main (String[] args) throws IOException {  
  
        Socket s = new Socket("localhost", 8888);  
  
        Scanner in = new Scanner(s.getInputStream());  
        PrintWriter out = new PrintWriter(s.getOutputStream());  
  
        String command = "DEPOSIT 3 1000";  
        System.out.println("Sending: " + command);  
  
        out.println(command);  
        out.flush();  
  
        String response = in.nextLine();  
        System.out.println("Receiving: " + response);  
  
        command = "QUIT";  
        out.println(command);  
        out.flush();  
  
        s.close();  
    }  
}
```

Execution





Lecture 8

- Socket Programming
- Getting Web Data
 - java.net package
 - REST API

Application Layer Protocols

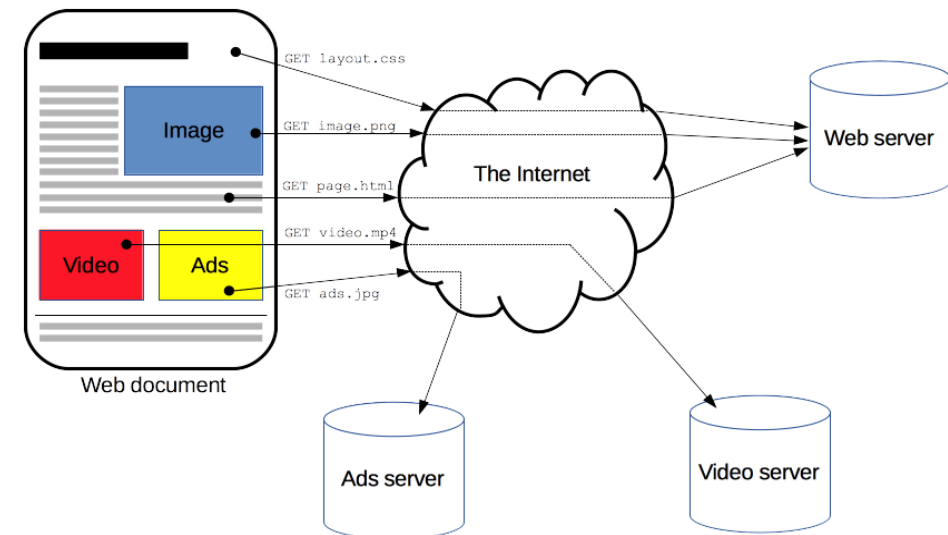
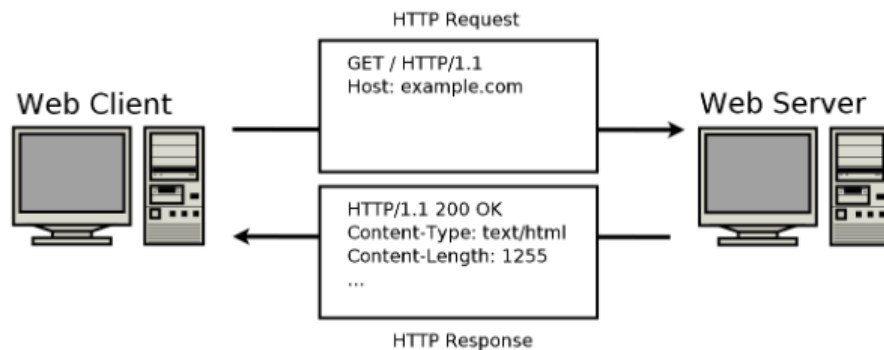
- Each Internet application has a different application protocol, which describes how the data for that particular application are transmitted.
- A port number helps a computer decide which application should receive an incoming piece of data

Well-known port numbers are reserved and we can no longer use them for other purposes

Port number	Protocol that uses it
21	File Transfer Protocol (FTP)
25	Simple Mail Transfer Protocol (SMTP)
80 & 8080	HyperText Transfer Protocol (HTTP)
110	Post Office Protocol v3 (POP3)
143	Internet Message Access Protocol (IMAP)
443	HyperText Transfer Protocol over SSL/TLS (HTTPS)
666	Doom Multiplayer game
989	Secure FTP (SFTP)
23	Telnet
25565	Minecraft Multiplayer Default Port
27015	Source Engine Multiplayer Default Port

HTTP (Hypertext Transfer Protocol)

- HTTP is a protocol for fetching resources such as HTML documents. It is the foundation of any data exchange on the Web
- It is a client-server protocol, which means requests are initiated by the client, usually the [web browser](#).
- [Web server](#) responds with an HTTP response



<https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview>

HTTP Request Commands

Table 1 HTTP Commands

Command	Meaning
GET	Return the requested item
HEAD	Request only the header information of an item
OPTIONS	Request communications options of an item
POST	Supply input to a server-side command and return the result
PUT	Store an item on the server
DELETE	Delete an item on the server
TRACE	Trace server communication

http://www.tcpipguide.com/free/t_HTTPResponseMessageFormat.htm

	Request Line
GET /index.html HTTP/1.1	
Date: Thu, 20 May 2004 21:12:55 GMT	General Headers
Connection: close	
Host: www.myfavoriteamazingsite.com	
From: joeblow@somewebsitesomewhere.com	Request Headers
Accept: text/html, text/plain	
User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)	Entity Headers
	Message Body

HTTP Response	
HTTP/1.1 200 OK	Status Line
Date: Thu, 20 May 2004 21:12:58 GMT	General Headers
Connection: close	
Server: Apache/1.3.27	Response Headers
Accept-Ranges: bytes	
Content-Type: text/html	Entity Headers
Content-Length: 170	
Last-Modified: Tue, 18 May 2004 10:14:49 GMT	
<pre> <html> <head> <title>Welcome to the Amazing Site!</title> </head> <body> <p>This site is under construction. Please come back later. Sorry!</p> </body> </html> </pre>	
Message Body	

HTTP Request/Response Message Format

Fetching a web page with socket

```
public static void readBySocket(String host) throws IOException {  
    try(Socket s = new Socket(host, 80)){  
        InputStream instream = s.getInputStream();  
        OutputStream outstream = s.getOutputStream();  
  
        // working with text  
        Scanner in = new Scanner(instream);  
        PrintWriter out = new PrintWriter(outstream);  
  
        // Send command  
        String resource = "/";  
        String command = "GET " + resource + " HTTP/1.1\n" +  
            "Host: " + host + "\n";  
        System.out.println(command);  
        out.println(command);  
        out.flush();  
  
        // Read server response  
        while(in.hasNextLine()){  
            String input = in.nextLine();  
            System.out.println(input);  
        }  
    }  
}
```

Problems

- We have to handle socket connections and socket errors by ourselves
- We have to manually create HTTP requests with the correct format
- We have to manually parse HTTP responses

To access web servers in Java, we want to work **at a higher level** than socket connections and HTTP requests

java.net package

Provides the classes for implementing networking applications.

The java.net package can be roughly divided in two sections:

- *A Low Level API*, which deals with the following abstractions:
 - *Addresses*, which are networking identifiers, like IP addresses.
 - *Sockets*, which are basic bidirectional data communication mechanisms.
 - *Interfaces*, which describe network interfaces.
- *A High Level API*, which deals with the following abstractions:
 - *URIs*, which represent Universal Resource Identifiers.
 - *URLs*, which represent Universal Resource Locators.
 - *Connections*, which represents connections to the resource pointed to by *URLs*.

https://docs.oracle.com/javase/7/docs/api/java/net/package-summary.html#package_description



Fetching a web page with URLConnection

```
public static void readByURLConnection(String url) throws IOException {
    URL u = new URL(url);
    // Open connection
    URLConnection conn = u.openConnection();
    // For HTTP an HttpURLConnection will be returned
    HttpURLConnection httpConn = (HttpURLConnection) conn;

    // Check response code and status
    int code = httpConn.getResponseCode();
    String msg = httpConn.getResponseMessage();
    System.out.println(code + " " + msg);
    if (code != HttpURLConnection.HTTP_OK) {
        return;
    }

    // Read server response
    InputStream istream = httpConn.getInputStream();
    Scanner in = new Scanner(istream);
    while (in.hasNextLine()) {
        System.out.println(in.nextLine());
    }
}
```




Lecture 8

- Socket Programming
- Getting Web Data
 - java.net package
 - REST API

REST API IN ACTION

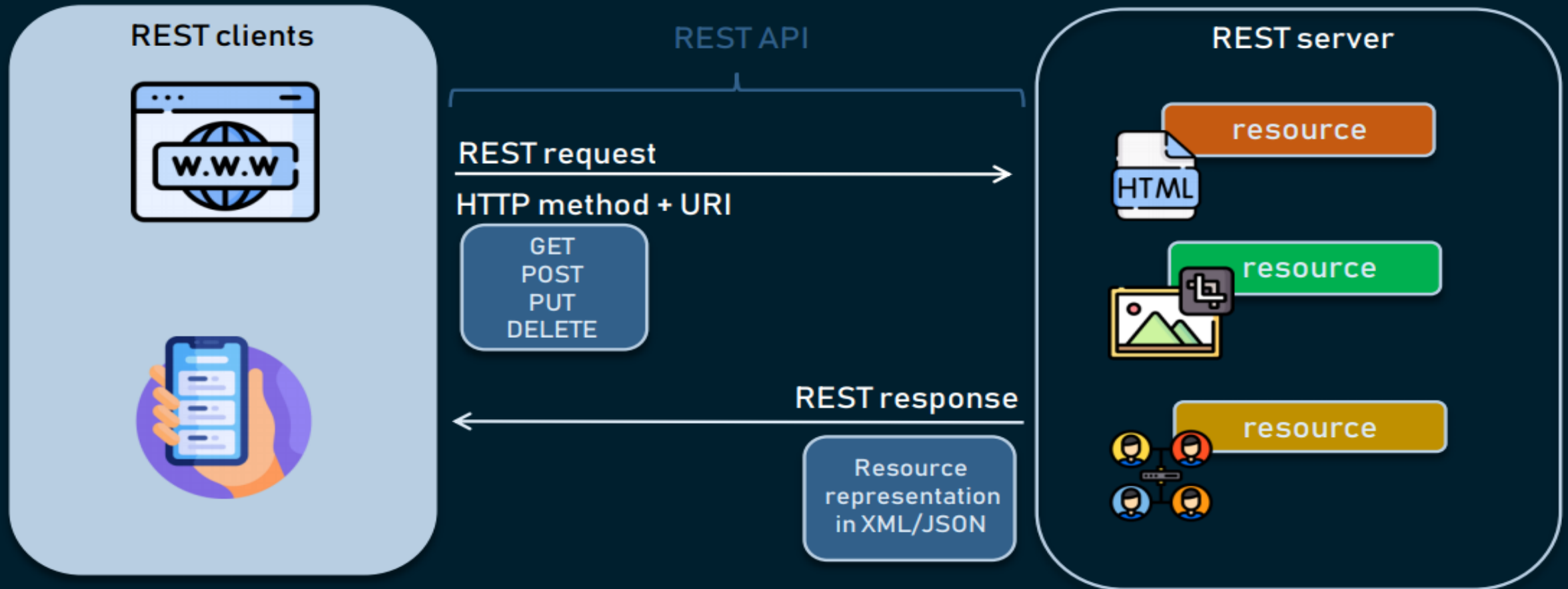


Image source: <https://www.altexsoft.com/blog/rest-api-design/>

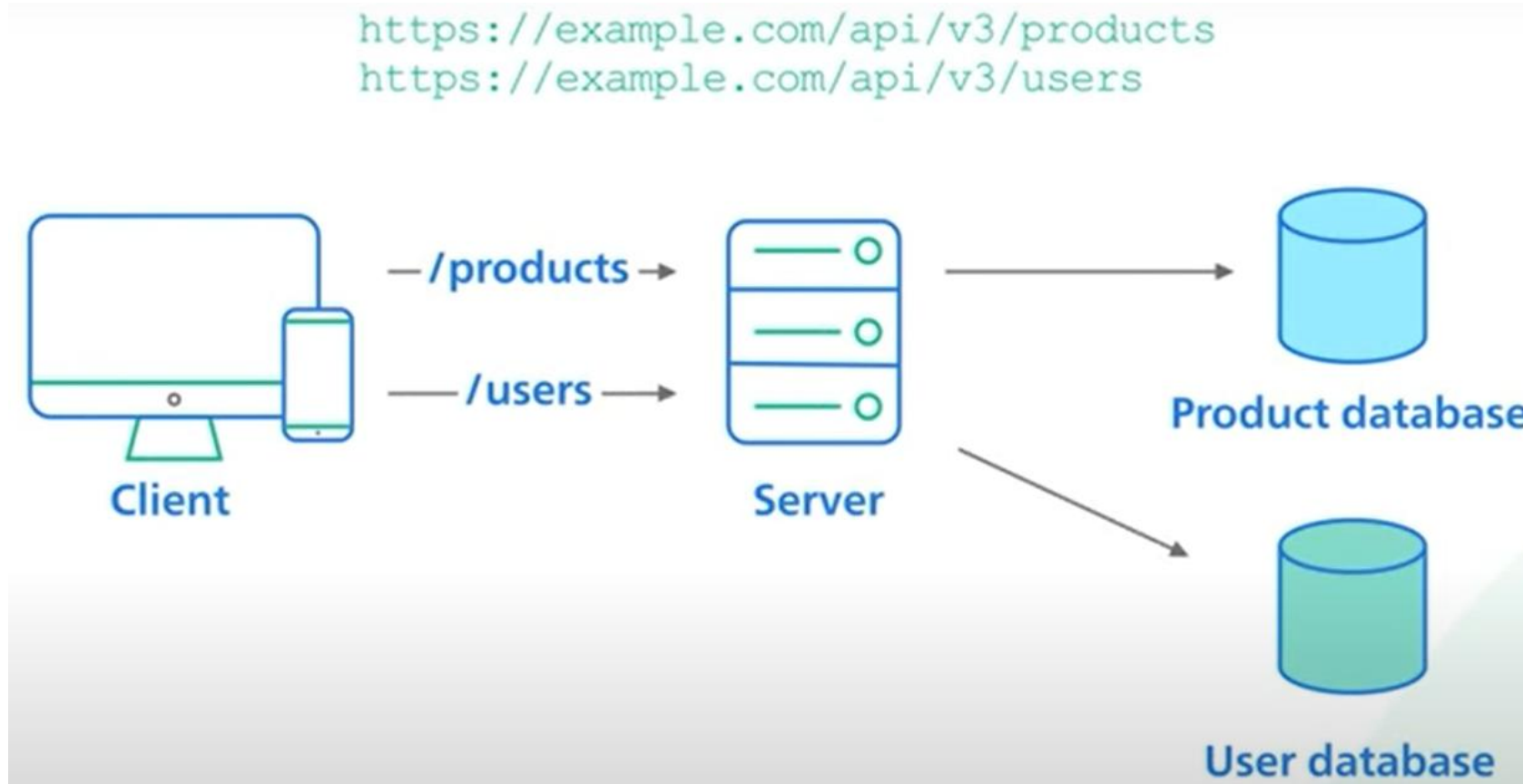
RESTful API Request Design

Request = Verb + Resources

GET
PUT
PATCH
POST
DELETE

- Typically use **noun** in plural form indicating the resources, e.g., questions
- Allow parameters for filtering, e.g., ?limit=10

RESTful API Request Design



Using **nouns** to differentiate different resources

GitHub REST API

URL: <https://api.github.com/>

Documentation: <https://docs.github.com/en/rest>

GET `/repos/{owner}/{repo}`

Get a repository info by its owner and repo name

<https://docs.github.com/en/rest/repos/repos#get-a-repository>

GET `/repos/{owner}/{repo}/contributors`

List repository contributors

POST `/repos/{owner}/{repo}/issues`

Create an issue (must have pull access to this repo)

PATCH `/repos/{owner}/{repo}/releases/{release_id}`

Update a release (must have push access to this repo)

GET `/search/topics`

Search for topics (should specify the topic using parameters)

Stack Overflow REST API

<https://api.stackexchange.com/docs>

Try It

Stack Overflow [\[edit\]](#)

[link](#) | [default filter](#) [\[edit\]](#) ▼

page	999	pagesize	999	fromdate	31
todate	31	order	desc	min	31
max	31	sort	activity	ids	27872387

</2.3/questions/27872387?order=desc&sort=activity&site=stackoverflow>

Run

```
{
  "items": [
    {
      "tags": [
        "java",
        "lambda",
        "java-8"
      ],
      "owner": {
        "account_id": 2079767,
        "reputation": 6280,
        "user_id": 1852033,
        "user_type": "registered",
        "accept_rate": 77,
        "profile_image": "https://www.gravatar.com/avatar/89b9541e85fd0534e3871ad790e651f1?s=256&d=i",
        "display_name": "Leo Ufimtsev",
        "link": "https://stackoverflow.com/users/1852033/leo-ufimtsev"
      },
      "is_answered": true,
      "view_count": 189988,
      "accepted_answer_id": 27872395,
      "answer_count": 8,
      "score": 213,
    }
  ]
}
```

Stack Overflow REST API

REST Service URL

Requested resource

Parameter

```
String s= "https://api.stackexchange.com/questions/27872387?site=stackoverflow";  
URL url = new URL(s);      java.net package  
URLConnection conn = (URLConnection)url.openConnection();  
conn.setRequestMethod("GET"); Request verb  
conn.connect();  
  
int responseCode = conn.getResponseCode();      200  
String responseMessage = conn.getResponseMessage();      OK  
String contentEncoding = conn.getContentEncoding();      gzip
```

Request Response

HTTP Status Code

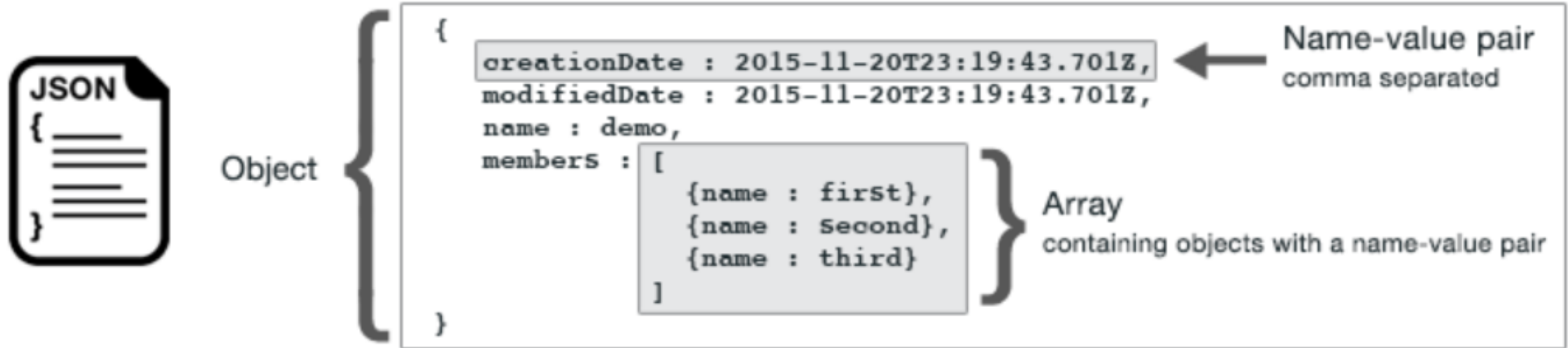


GET /repos/{owner}/{repo}

Status: 200 OK

```
{
  "id": 1296269,
  "node_id": "MDEwO1JlcG9zaXRvcnkxMjk2MjY5",
  "name": "Hello-World",
  "full_name": "octocat/Hello-World",
  "owner": {
    "login": "octocat",
    "id": 1,
    "node_id": "MDQ6VXNlcjE=",
    "avatar_url": "https://github.com/images/error/octocat_happy.gif",
    "gravatar_id": "",
    "url": "https://api.github.com/users/octocat",
    "html_url": "https://github.com/octocat",
    "followers_url": "https://api.github.com/users/octocat/followers",
    "following_url": "https://api.github.com/users/octocat/following{/other_user}",
    "gists_url": "https://api.github.com/users/octocat/gists{/gist_id}",
```

JSON format



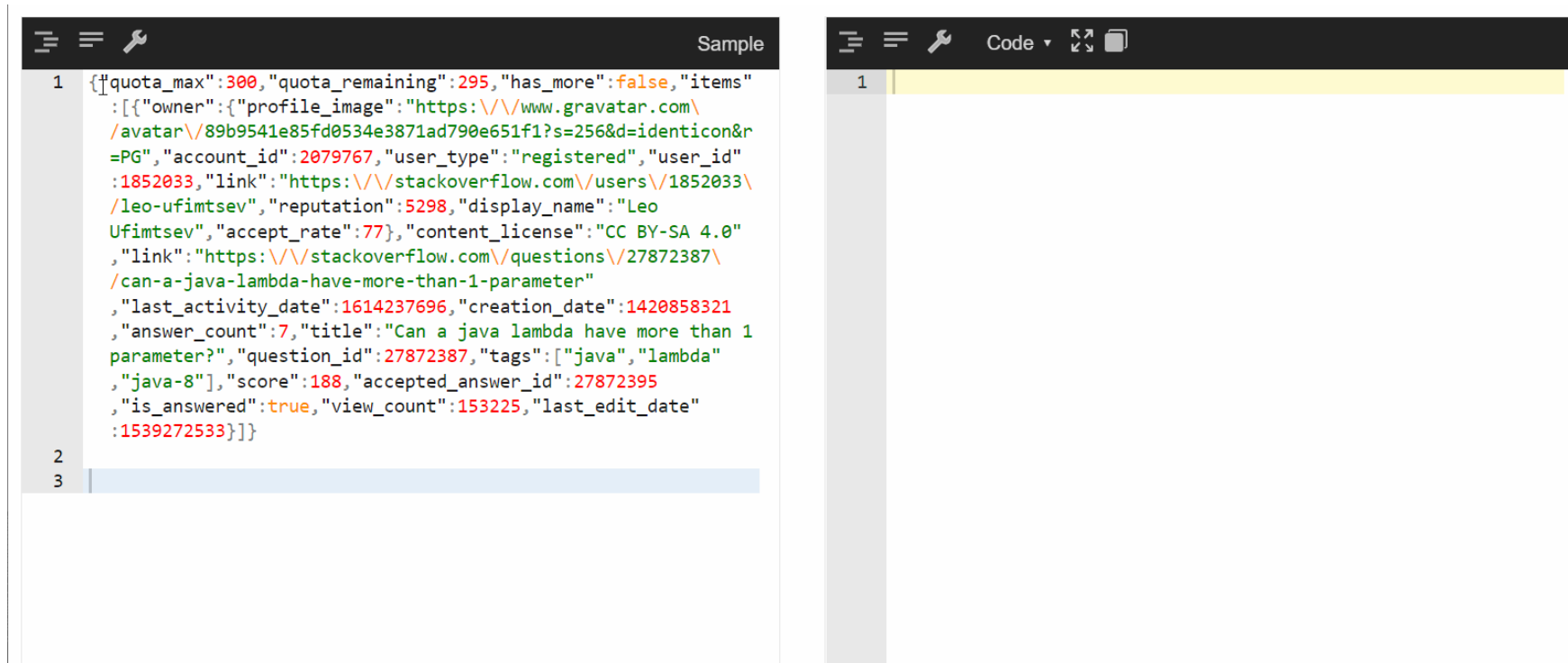
TAO Yida@SUSTECH

JSON

- JavaScript Object Notation
- An open data interchange format that is both human and machine-readable
- Independent of any programming language

JSON Helper Tools

- Java Libraries for parsing and creating JSON string: JSON-simple, GSON, Jackson, etc.
- JSON viewers (help formatting the JSON string)



Type <https://api.github.com/repos/spring-projects/spring-boot> in your browser and see what happens

Next Lecture

- GUI Intro
- JavaFX